

DATA STRUCTURE LAB

(DCO – 414)

Practical No. 6 Write a program to create a linked list by inserting nodes at:

i). A particular position in the linked list ii). After a key value in the linked list.

b). Write a program to delete a node from a linked list from:

i). A particular position in the linked list ii). After a key value in the linked list

Submitted by:

NAME-HASAN SIDDIQUE

ROLL NO-24DCS015

Diploma in Computer Engineering 4th Semester



Date of Assign – 16/3/2026

Date of Checking – 30/3/2026

University Polytechnic,

Faculty of Engineering and Technology

Jamia Millia Islamia

New Delhi – 110025, Session 2025-26

-----ALGORITHM-----

STEP 1: Start

STEP 2: Create a linked list (initially head = NULL)

Insertion at a Particular Position

STEP 3: Read value and position

STEP 4: Create a new node and assign value

STEP 5: If list is empty OR position = 1

→ Set new node next = head

→ Update head = new node

STEP 6: Else traverse the list up to (position - 1) node

STEP 7: Set new node next = current node next

STEP 8: Set current node next = new node

Insertion After a Key Value

STEP 9: Read value and key

STEP 10: Create a new node and assign value

STEP 11: If list is empty

→ Set head = new node

STEP 12: Else traverse the list to find node with given key

STEP 13: If key is found

→ Set new node next = current node next

→ Set current node next = new node

STEP 14: Else traverse to last node

→ Insert new node at end

Deletion from a Particular Position

STEP 15: Read position

STEP 16: If list is empty

→ Stop

STEP 17: If position = 1

→ Store head in temp

→ Update head = head next

→ Free temp

STEP 18: Else traverse the list up to (position - 1) node

STEP 19: Store node to be deleted in temp

STEP 20: Set current node next = temp next

STEP 21: Free temp

Deletion After a Key Value

STEP 22: Read key

STEP 23: If list is empty

→ Stop

STEP 24: Traverse the list to find node with given key

STEP 25: If key is found AND next node exists

→ Store next node in temp

→ Set current node next = temp next

→ Free temp

STEP 26: Stop

-----CODE-----

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void display() {
    if (head == NULL) {
        printf("\nList is empty\n");
        return;
    }
    struct Node* temp = head;
    printf("\nList: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void insertAtPosition(int value, int pos) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    // Case 1: Insert at beginning
    if (pos <= 1 || head == NULL) {
        newNode->next = head;
        head = newNode;
        printf("\nInserted at beginning (handled edge case)\n");
        return;
    }
}
```

```

    }
    struct Node* temp = head;
    int i = 1;
    while (temp->next != NULL && i < pos - 1) {
        temp = temp->next;
        i++;
    }
    newNode->next = temp->next;
    temp->next = newNode;
    printf("\nInserted at position %d (or last if out of range)\n", pos);
}

void insertAfterKey(int value, int key) {
    if (head == NULL) {
        printf("\nList is empty, inserting as first node\n");
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = value;
        newNode->next = NULL;
        head = newNode;
        return;
    }
    struct Node* temp = head;
    while (temp != NULL && temp->data != key) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("\nKey not found, inserting at end\n");
        temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        newNode->data = value;
        newNode->next = NULL;
    }
}

```

```

        temp->next = newNode;
        return;
    }
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = temp->next;
    temp->next = newNode;
    printf("\nInserted after key %d\n", key);
}

void deleteAtPosition(int pos) {
    if (head == NULL) {
        printf("\nList is empty, nothing to delete\n");
        return;
    }
    if (pos <= 1) {
        struct Node* temp = head;
        head = head->next;
        free(temp);
        printf("\nDeleted first node\n");
        return;
    }
    struct Node* temp = head;
    int i = 1;
    while (temp->next != NULL && i < pos - 1) {
        temp = temp->next;
        i++;
    }
    if (temp->next == NULL) {
        printf("\nPosition out of range\n");
        return;
    }
}

```

```

    struct Node* del = temp->next;
    temp->next = del->next;
    free(del);

    printf("\nDeleted node at position %d\n", pos);
}

void deleteAfterKey(int key) {
    if (head == NULL) {
        printf("\nList is empty\n");
        return;
    }

    struct Node* temp = head;

    while (temp != NULL && temp->data != key) {
        temp = temp->next;
    }

    if (temp == NULL) {
        printf("\nKey not found\n");
        return;
    }

    if (temp->next == NULL) {
        printf("\nNo node exists after key\n");
        return;
    }

    struct Node* del = temp->next;
    temp->next = del->next;
    free(del);

```

```

    printf("\nDeleted node after key %d\n", key);
}

void main() {
    int choice, value, pos, key;

    while (1) {
        printf("\n\n--- MENU ---\n");
        printf("1. Insert at position\n");
        printf("2. Insert after key\n");
        printf("3. Delete at position\n");
        printf("4. Delete after key\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                printf("Enter position: ");
                scanf("%d", &pos);
                insertAtPosition(value, pos);
                break;

            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                printf("Enter key: ");
                scanf("%d", &key);
                insertAfterKey(value, key);
                break;

```


case 3:

```
printf("Enter position: ");  
scanf("%d", &pos);  
deleteAtPosition(pos);  
break;
```

case 4:

```
printf("Enter key: ");  
scanf("%d", &key);  
deleteAfterKey(key);  
break;
```

case 5:

```
display();  
break;
```

case 6:

```
exit(0);
```

default:

```
printf("\nInvalid choice\n");
```

```
}
```

```
}
```

```
}
```

-----OUTPUT-----

--- MENU ---

1. Insert at position
2. Insert after key
3. Delete at position
4. Delete after key
5. Display
6. Exit

Enter choice: 1

Enter value: 21

Enter position: 0

Inserted at beginning (handled edge case)

--- MENU ---

1. Insert at position
2. Insert after key
3. Delete at position
4. Delete after key
5. Display
6. Exit

Enter choice: 2

Enter value: 34

Enter key: 21

Inserted after key 21

--- MENU ---

1. Insert at position
2. Insert after key

3. Delete at position

4. Delete after key

5. Display

6. Exit

Enter choice: 5

List: 21 -> 34 -> NULL

--- MENU ---

1. Insert at position

2. Insert after key

3. Delete at position

4. Delete after key

5. Display

6. Exit

Enter choice: 6